

Kaito Ren, the reckless one, climbs first and thinks later.

Jun Takeda, swordsman whose Foresight Eye sees future moves.

Tomo Kuro, sleepy owl-keeper and strategist on the radio.

Gennai, a wandering monk who taught Kaito to take risks.

Mira Hoshi, map reader and medic, keeps the team honest.

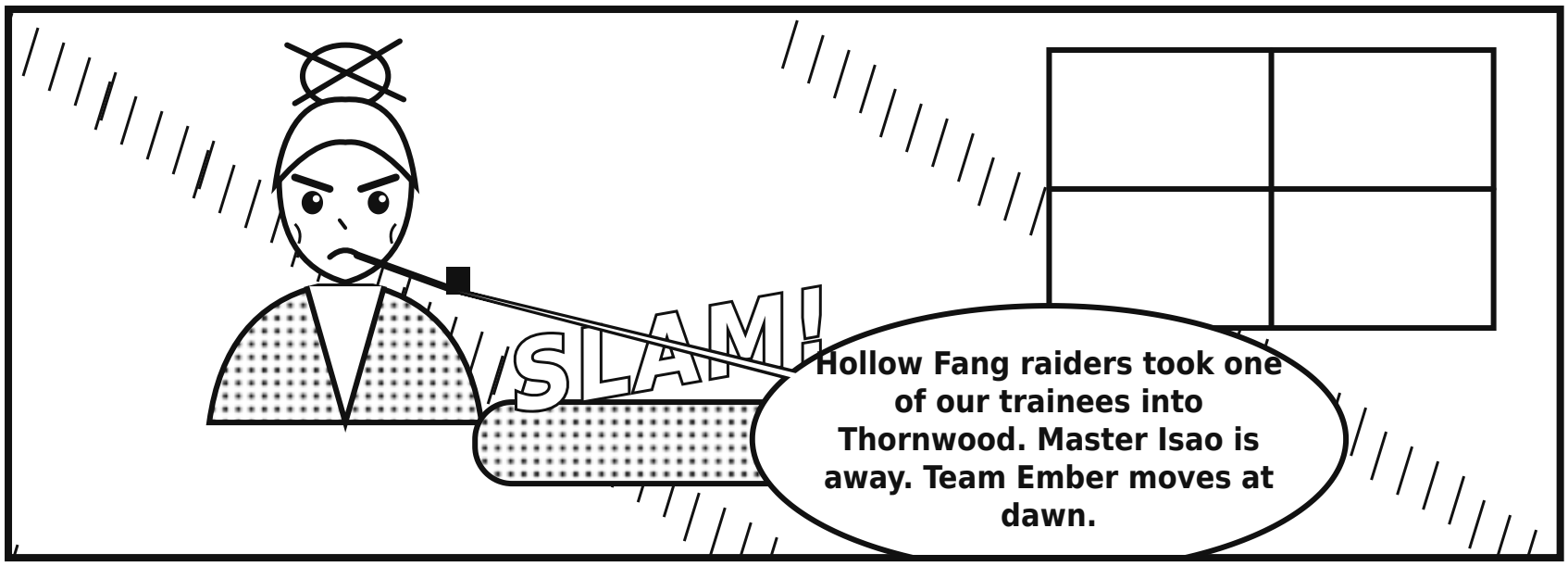
Rin Asano, seed-weaver who grows answers over generations.

Elder Sora, head of Hollow Pine Village.

Vesper, raider of the Hollow Fang, master of Stone Drop.

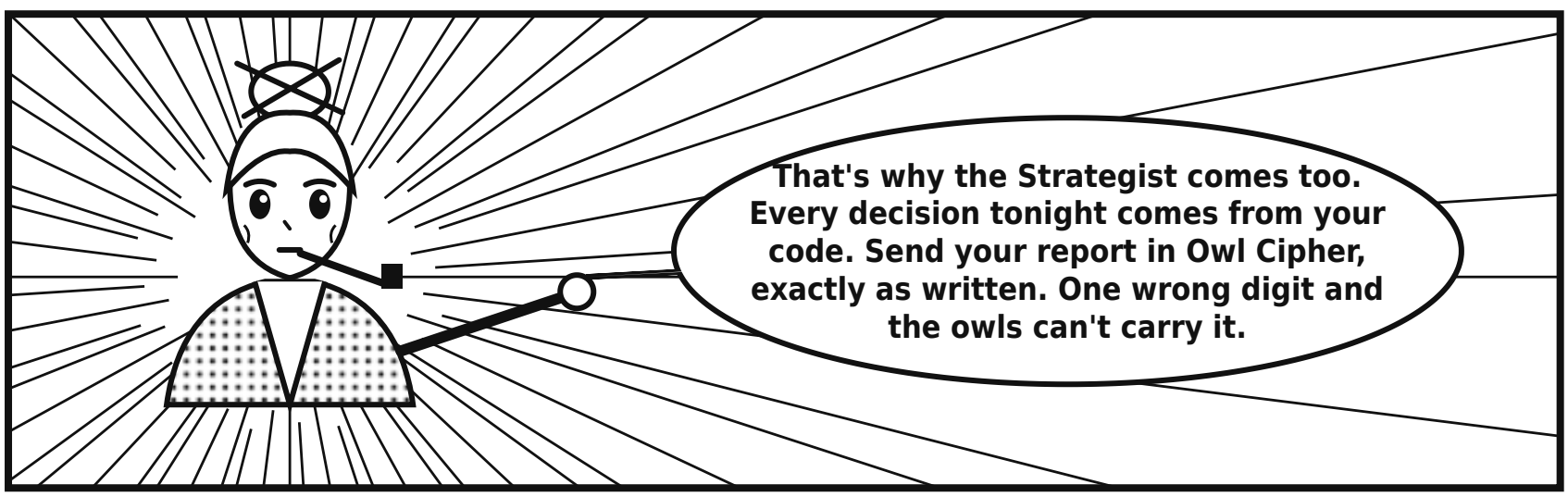
Chapter 0: The Mission

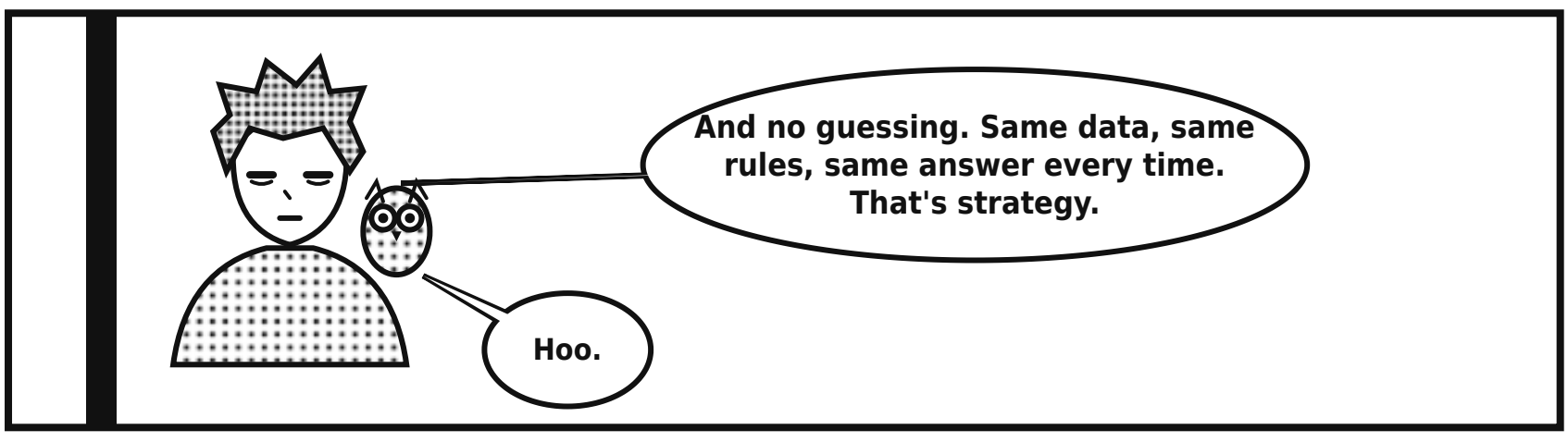
0











0 Scroll Box 0: Rules of the mission

These rules apply to all four missions. A mission's own Scroll Box adds to them and never replaces them.

General rules for every mission

1. Use Python 3.10 or newer and the standard library only (`math`, `random`, `itertools`). Submit one file, `thornwood.py`. It takes no input and prints only the Owl Cipher report from Scroll Box 5.
2. Compute every answer when the program runs. A hard-coded answer scores 0 for that mission. Type the data from the Scroll Boxes into your program exactly as given.
3. For randomness, create `rng = random.Random(seed)` with the seed the mission gives, and call only `rng.random()`. Never call `random.random()`, `randint`, `choice` or `shuffle`.
4. To draw a whole number from 0 to k − 1, use `int(rng.random() * k)`. Make the random draws in exactly the order described, and make no others.
5. Apply every tie-break rule exactly. Tie-breaks are what make every correct result identical.
6. Write each algorithm as its own function with a docstring.
7. Your program must finish within 5 minutes on a lab machine.

The four missions

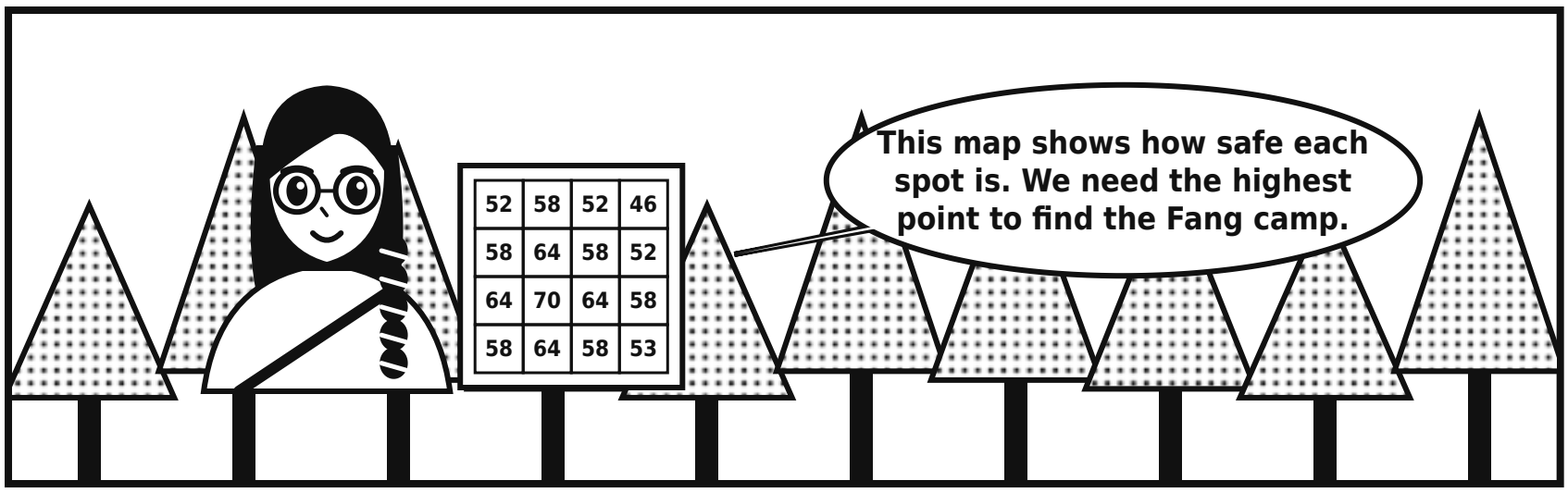
Chapter	Story	AI technique
1. The false summit	Climb to the safest lookout in Thornwood	Hill climbing, random restart, simulated annealing
2. Rin's seedcraft	Pack the best tool pouch under two limits	Genetic algorithm, checked by brute force
3. Eyes that see the future	Beat Vesper at Stone Drop	Minimax, full and depth-limited
4. Running on empty	Win the same duel with less effort, then pass Master Hyo's trial tree	Alpha-beta pruning: three Stone Drop runs and a tree with given terminal utilities

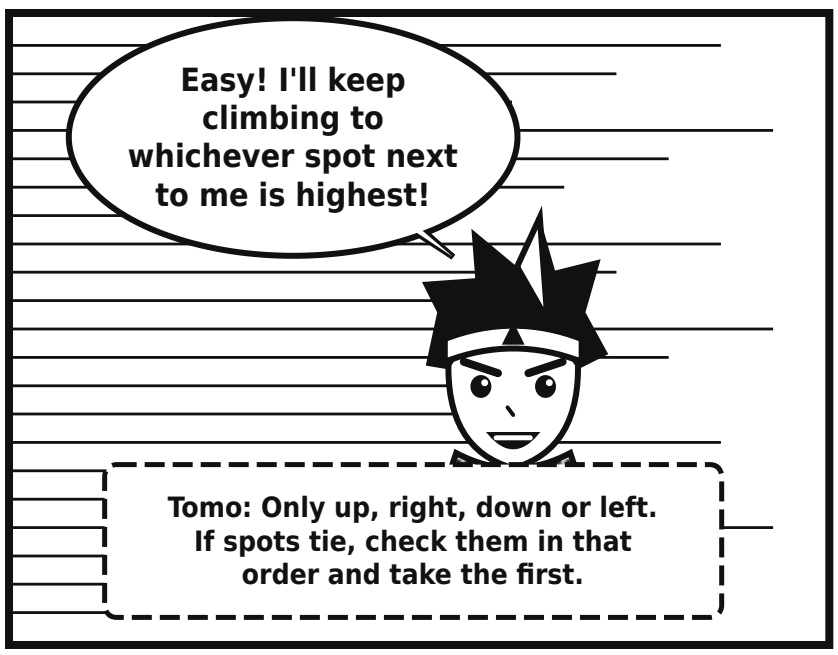
Submission

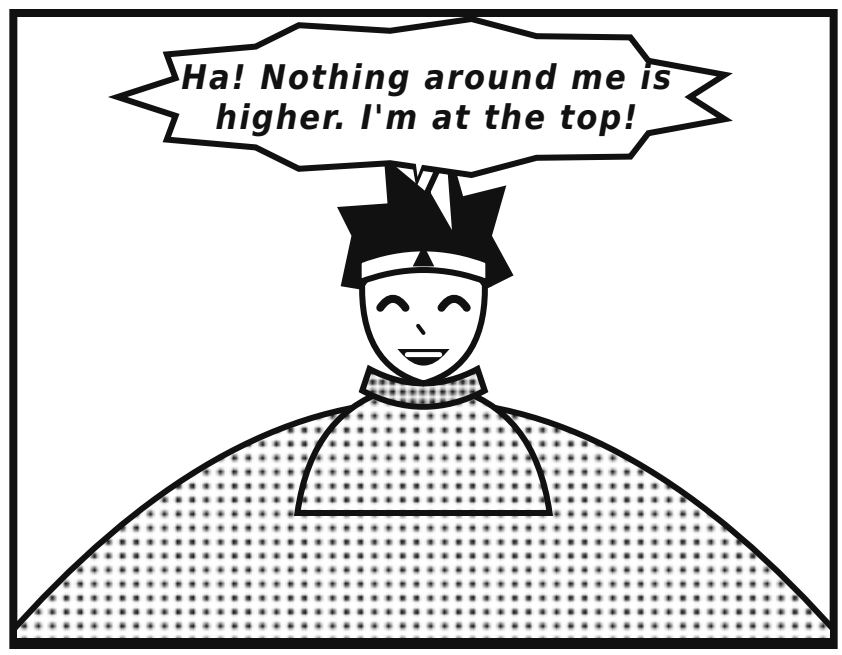
Submit `thornwood.py` and `debrief.pdf`, a report of at most four pages answering the questions in Chapter 5. Scroll Box 5 gives the exact report format.

Chapter 1: The False Summit

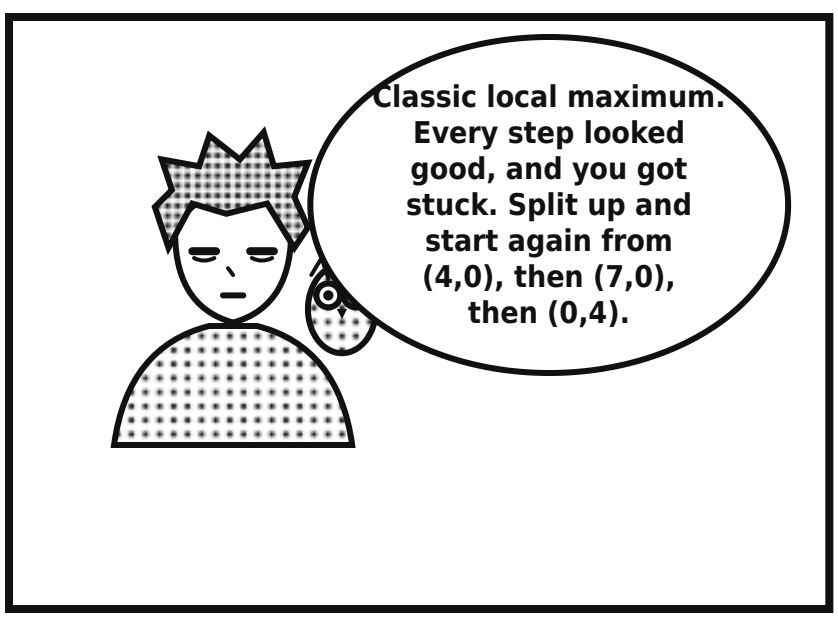
1

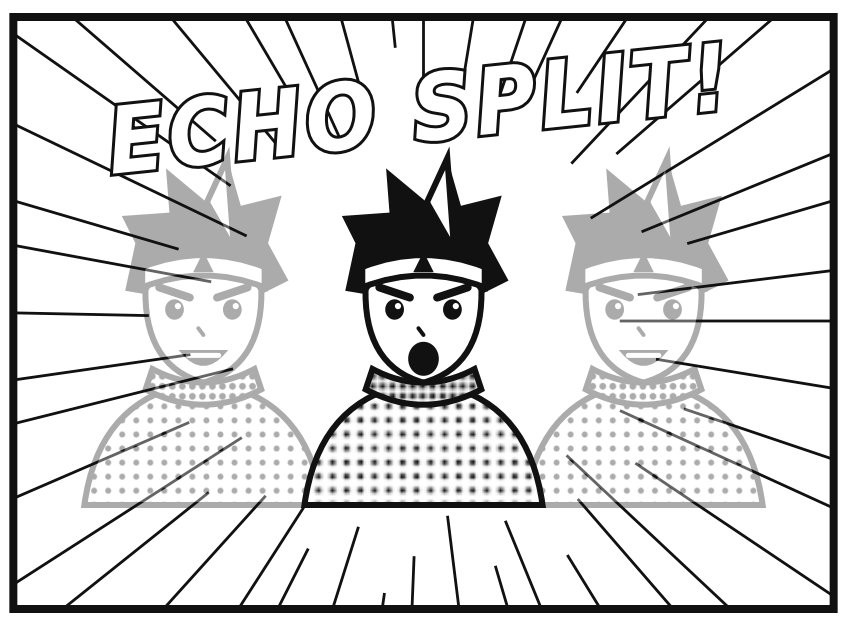


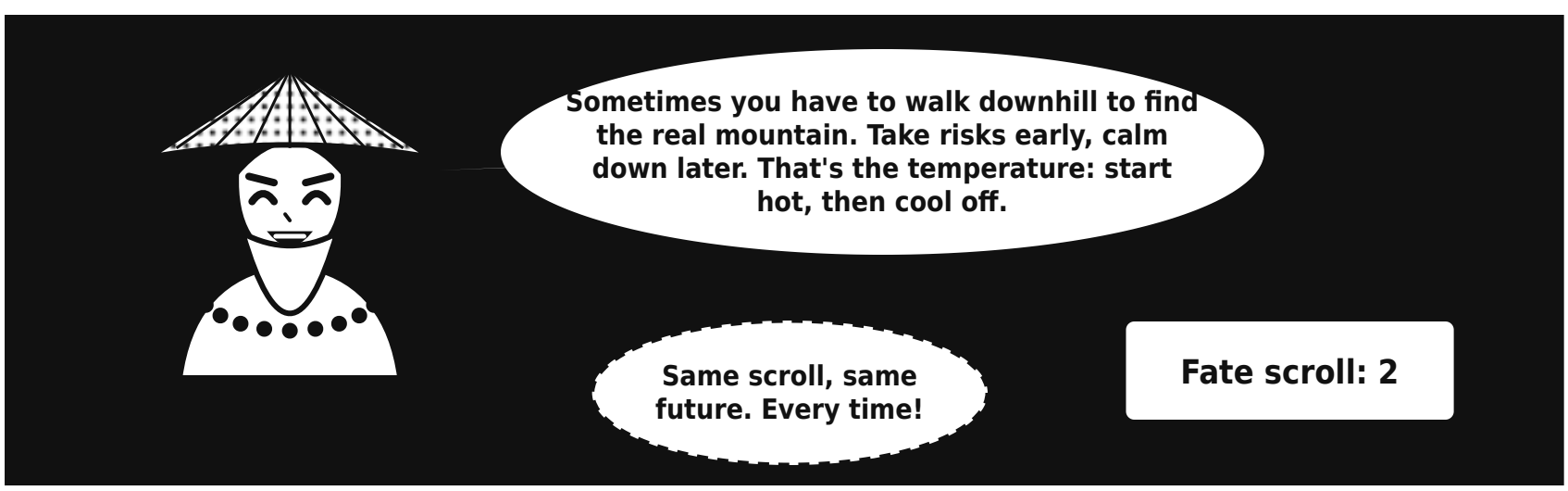












Read the panels left to right, top to bottom.

1 Scroll Box 1: Lookout climb

Find the safest lookout with hill climbing, random restart and simulated annealing.

The safety map

Cells are written as (row, column). Row 0 is the top row and column 0 is the left column.

row \ col	0	1	2	3	4	5	6	7
0	52	58	52	46	40	46	53	46
1	58	64	58	52	46	58	60	53
2	64	70	64	58	53	60	67	60
3	58	64	58	53	60	67	74	67
4	52	58	53	50	67	74	81	74
5	46	53	60	67	74	81	88	81
6	53	60	67	74	81	88	95	88
7	46	53	60	67	74	81	88	81

Neighbour order is always up, right, down, left. These are direction numbers 0, 1, 2 and 3. A neighbour outside the map does not exist.

Part A: Kaito's climb (steepest-ascent hill climbing)

1. Start at (0,0).
2. Look at every neighbour. Move to the one with the highest value, but only if it is strictly higher than the current cell. If several neighbours tie for highest, take the first one in neighbour order.
3. Each move counts as one step. Stop when no neighbour is higher.

Report the final cell, its value and the number of steps.

Part B: Echo Split (random restart)

Run Part A again from (4,0), then (7,0), then (0,4). Together with Kaito's climb from (0,0) that makes four climbs. Report the best final value of the four climbs, the start cell of the first climb that reached it, and the total number of steps across all four climbs.

Part C: Gennai's method (simulated annealing)

Create `rng = random.Random(2)`. Start at (0,0) with temperature T = 20.0. The best value starts as the value of (0,0). Run exactly 200 iterations:

1. Draw `d = int(rng.random() * 4)` and take the neighbour in direction d.
2. If that neighbour is outside the map, skip to step 4.
3. Let Δ = value(neighbour) − value(current). If Δ ≥ 0, move. If Δ < 0, draw `r = rng.random()` and move only if `r < math.exp(Δ / T)`. Draw r only when Δ < 0. Each move adds 1 to the accepted count. After a move, if the new value is strictly higher than the best value, it becomes the best value.
4. Set T = T × 0.97. This happens in every iteration, including skipped ones.

Report the final cell, the final value, the best value and the accepted count.

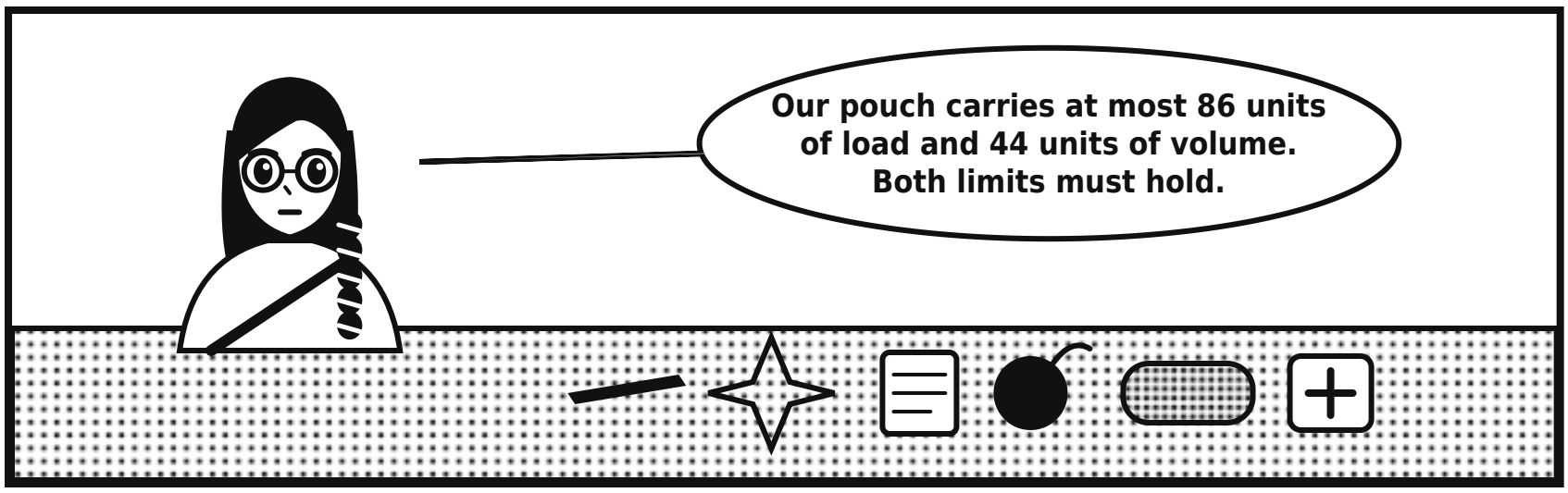
Part D: Fate test

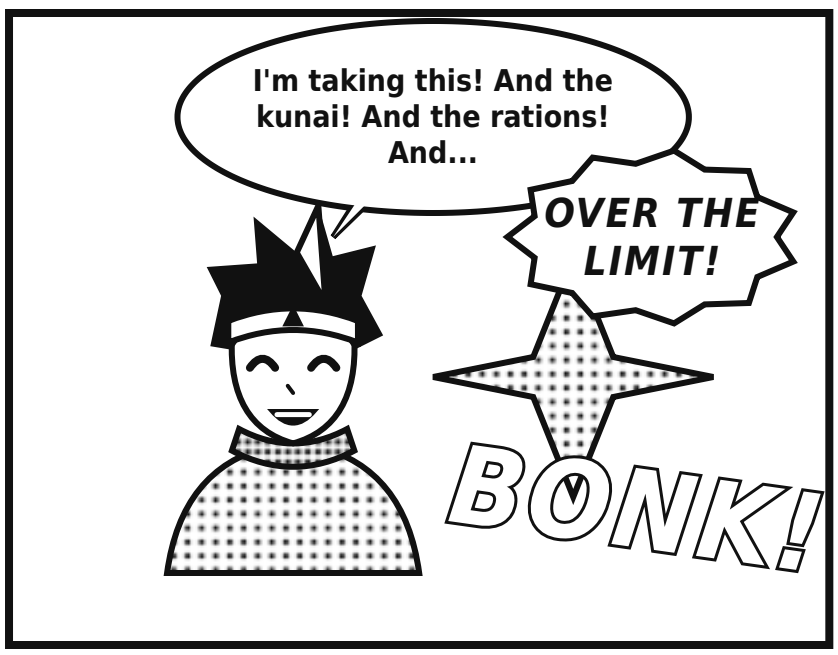
Repeat Part C with seeds 1, 2, 3 and so on up to 20. Report how many of these 20 runs end with a final value of 95.

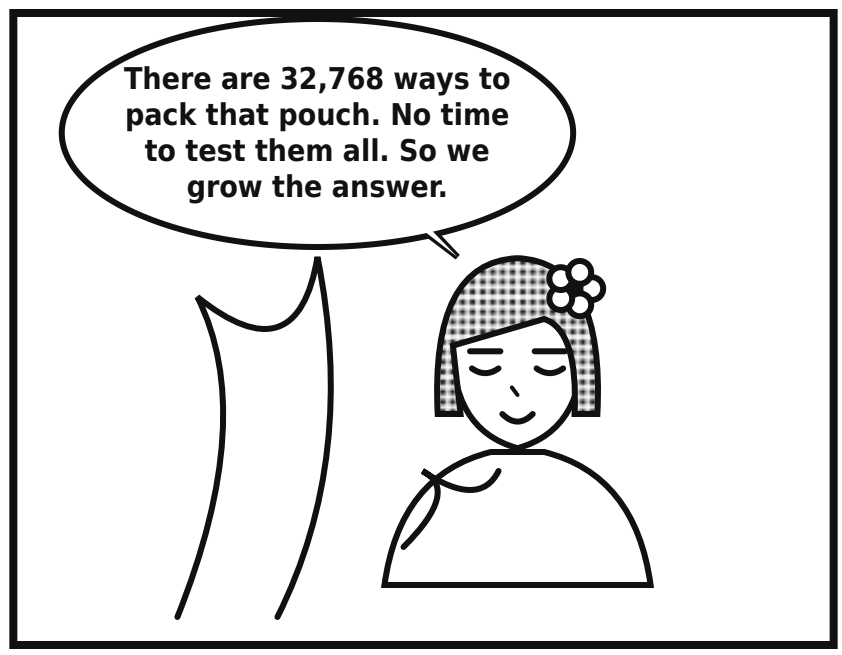
Report lines

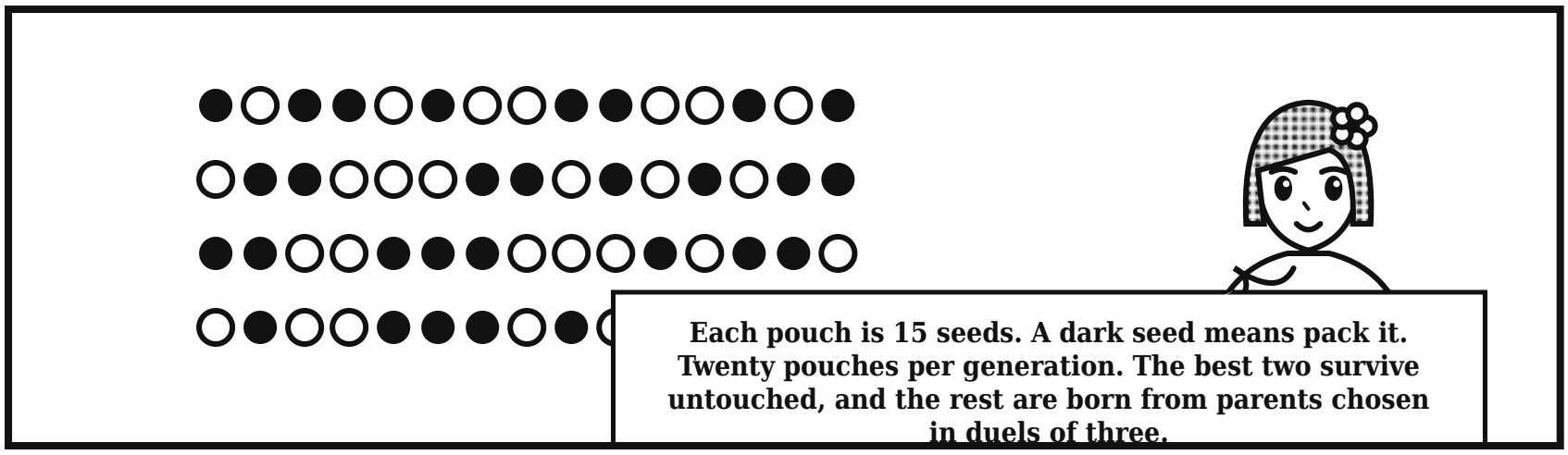
`HC_FINAL`, `HC_VALUE`, `HC_STEPS`, `RR_BEST_VALUE`, `RR_BEST_START`, `RR_TOTAL_STEPS`, `SA_FINAL`, `SA_FINAL_VALUE`, `SA_BEST_VALUE`, `SA_ACCEPTED` and `SA_SUCCESS_1_TO_20`, in the format of Scroll Box 5.

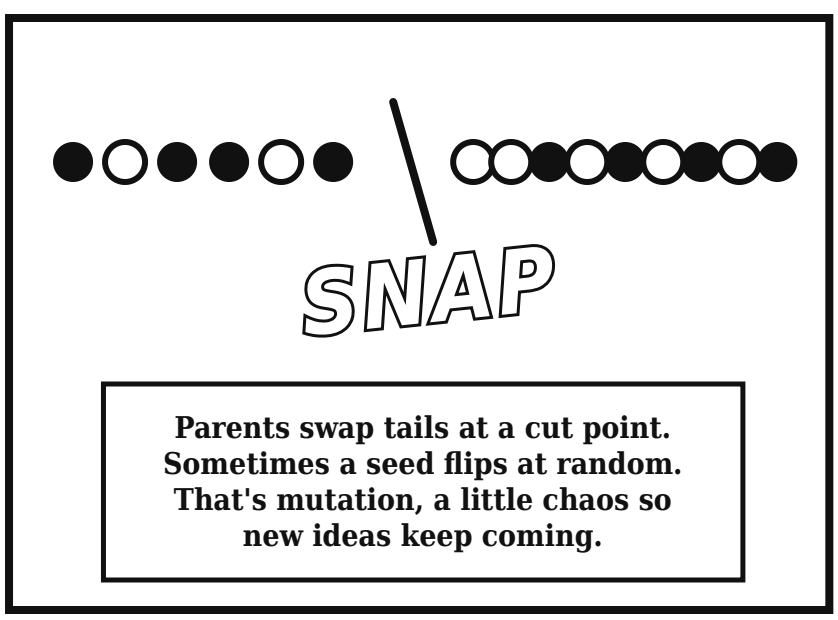
Chapter 2: Rin's Seedcraft















Read the panels left to right, top to bottom.

2 Scroll Box 2: The tool pouch

Grow the best pouch with a genetic algorithm, then check it against the true optimum.

The tools

The pouch has two limits: total load at most **86** and total volume at most **44**. Each tool is either packed once or left behind.

#	Tool	Load	Volume	Usefulness
0	Kunai set	18	9	36
1	Wire spool	11	8	9
2	Smoke bombs	12	5	10
3	Paper bombs	9	3	32
4	Energy pills	5	7	31
5	Flash tags	5	6	8
6	Messenger scroll	11	3	33
7	Giant shuriken	20	6	27
8	Spare rations	13	6	5
9	Wind shuriken	11	2	35
10	Twin blades	14	6	21
11	Caltrops	15	9	13
12	Medical kit	8	8	26
13	Grappling hook	10	9	19
14	Binding tags	10	7	38

Definitions

- A **chromosome** is a list of 15 bits. Bit k = 1 means tool k is packed. Print it as a 15-character string with bit 0 first, for example `110000000000001`.
- **Fitness** is the total usefulness of the packed tools if both limits hold, and 0 otherwise.
- The **population** is an ordered list of 20 chromosomes in positions 0 to 19.
- To **rank** a population, sort by fitness from highest to lowest. Ties go to the chromosome in the lower position.

Generation 0 (the initial population)

Type these 20 chromosomes into your program exactly. They are generation 0, in positions 0 to 19. No random draws are used to build them.

Position	Chromosome (bit 0 first)	Position	Chromosome (bit 0 first)
0	`110101101111101`	10	`011001010000111`
1	`100010101111010`	11	`000110000001011`
2	`010111011011001`	12	`111001000111110`
3	`000010110111000`	13	`001001000011010`
4	`010001001010000`	14	`011001110010001`
5	`110111110111011`	15	`011000010011110`
6	`000011100111110`	16	`111011001000011`
7	`111100000010000`	17	`110110010001011`
8	`111011111111111`	18	`000001000010100`
9	`001111100111111`	19	`001000111111000`

The algorithm

Create `rng = random.Random(12)`. Its first draw is the first tournament draw in step 4.

1. **Start.** The current population is generation 0 from the table above.
2. **Evaluate.** Rank the population. Record the best fitness, the average fitness (sum of the 20 fitness values divided by 20) and the top-ranked chromosome. If this is generation 50, stop.
3. **Elitism.** The next population starts with copies of rank 1, then rank 2.
4. **Selection.** Run a tournament for parent 1, then one for parent 2. A tournament draws three positions, each with `int(rng.random() * 20)`. Repeats are allowed. The highest fitness wins, and a tie goes to the one drawn first.
5. **Crossover.** Draw `rng.random()`. If it is below 0.9, draw `cut = int(rng.random() * 14) + 1`. Child 1 takes bits 0 to cut − 1 from parent 1 and the rest from parent 2. Child 2 takes bits 0 to cut − 1 from parent 2 and the rest from parent 1. Otherwise the children are copies of parent 1 and parent 2, and no cut is drawn.
6. **Mutation.** For child 1, go through bits 0 to 14 in order. Draw `rng.random()` for each bit and flip it if the draw is below 1/15. Append child 1 to the next population. Then do the same for child 2 and append it.
7. **Repeat** steps 4 to 6 until the next population holds 20 chromosomes. Each round adds two children and 18 places remain after elitism, so the last round always fits both. Then the next population becomes the current one; go to step 2.

Children must be new lists. Changing a child must never change a parent or another member.

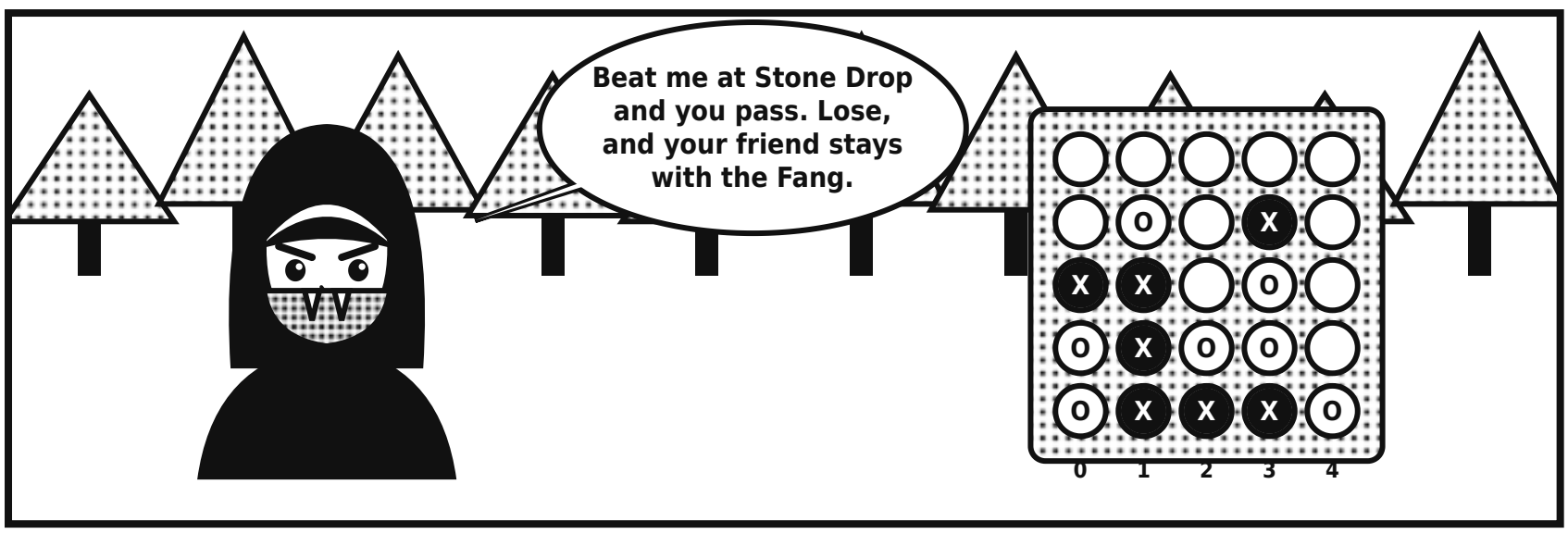
Tomo's check

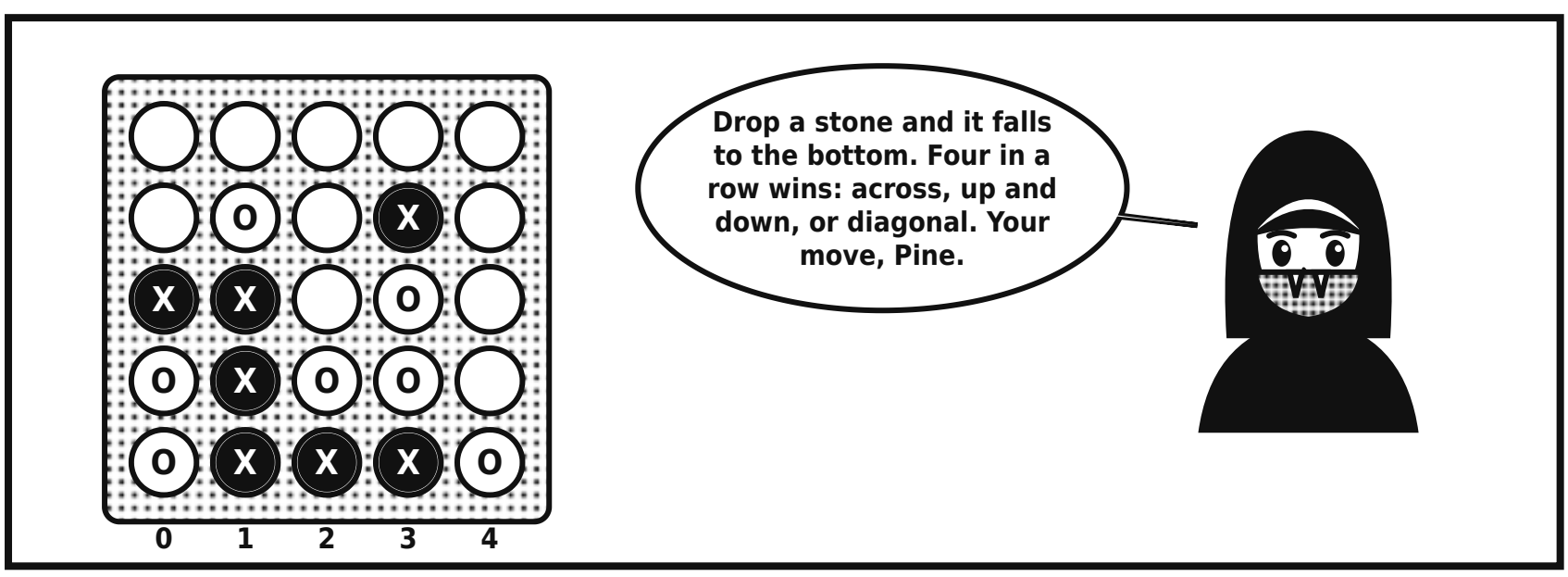
Find the true optimum exactly, by brute force over all 2^{15} = 32,768 pouches or by a dynamic program with both limits. Report its usefulness and its chromosome.

Report lines

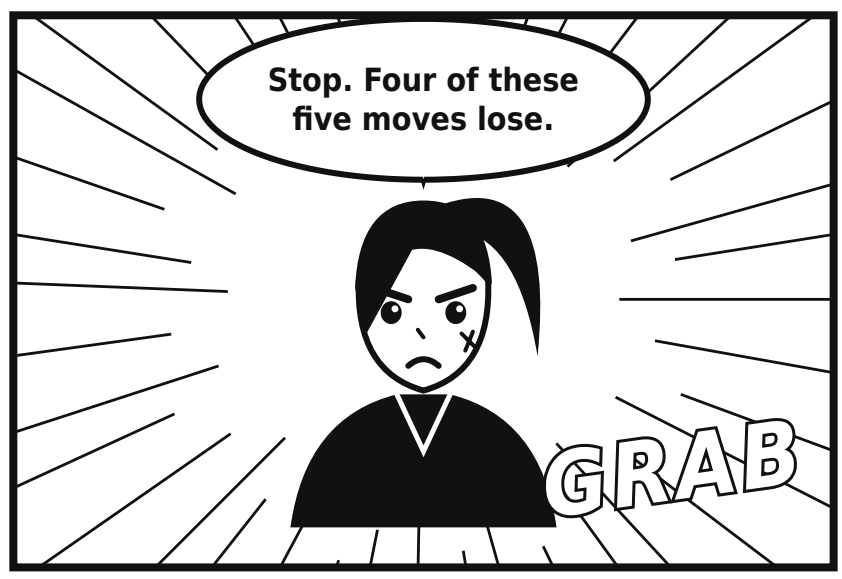
`GA_GEN_BEST` (best fitness at generations 0, 10, 20, 30, 40 and 50), `GA_BEST_CHROMOSOME` and `GA_BEST_FITNESS` (from generation 50), `GA_FIRST_GEN` (the earliest generation whose best fitness equals `GA_BEST_FITNESS`), `GA_FINAL_AVG` (generation 50, two decimals), `OPT_FITNESS` and `OPT_CHROMOSOME`.

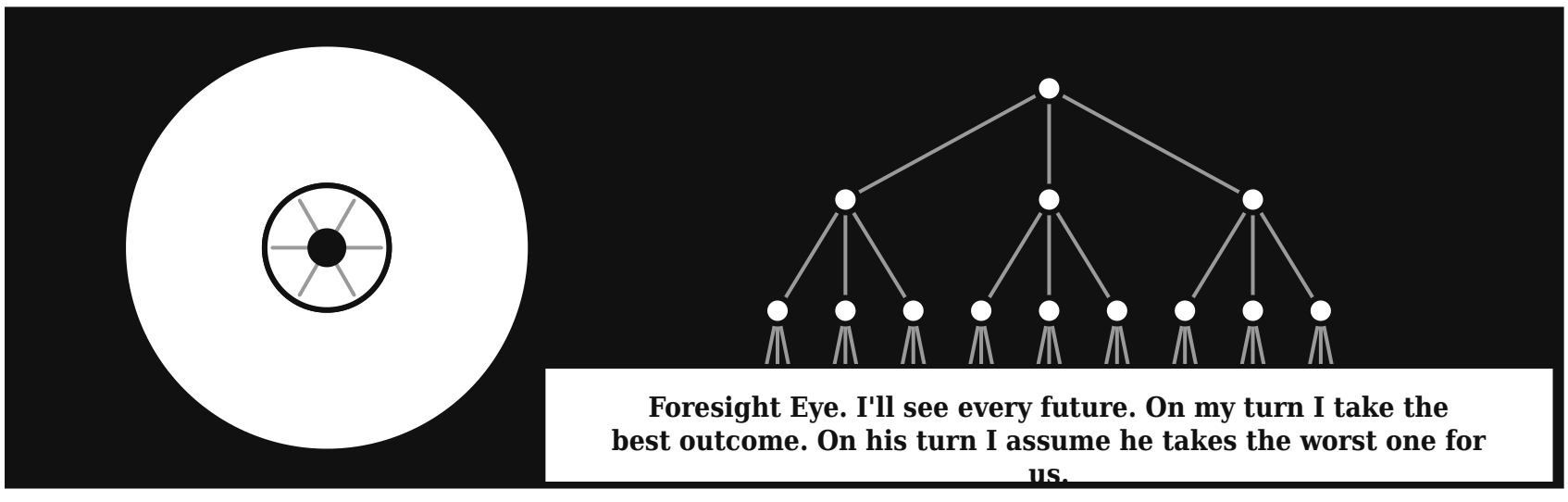
Chapter 3: Eyes That See the Future

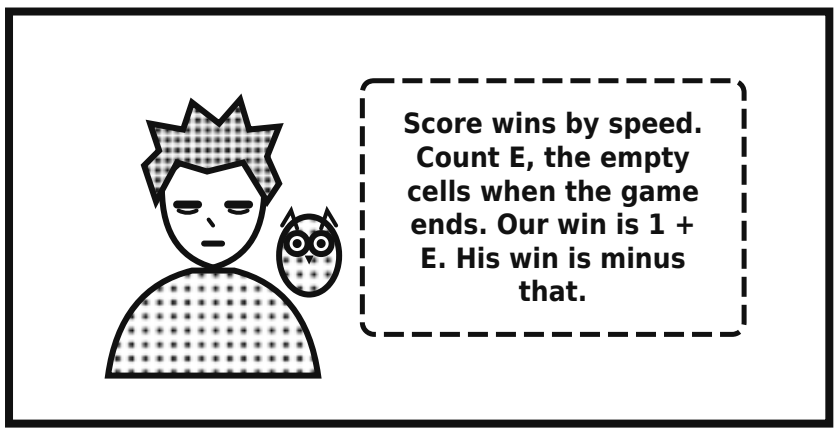


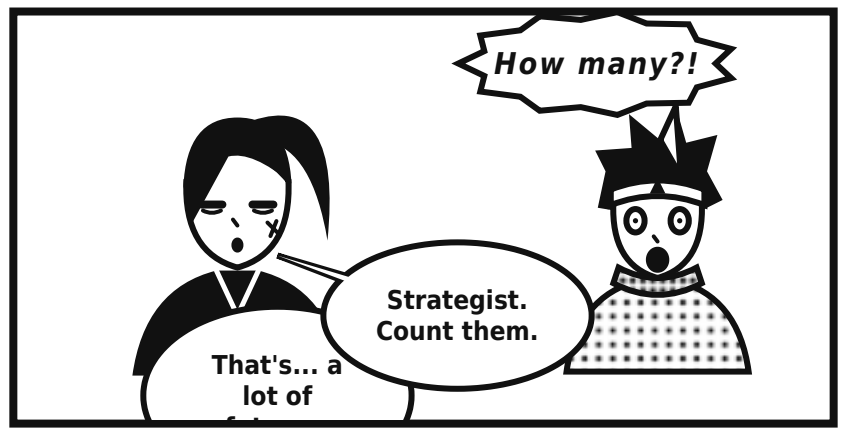


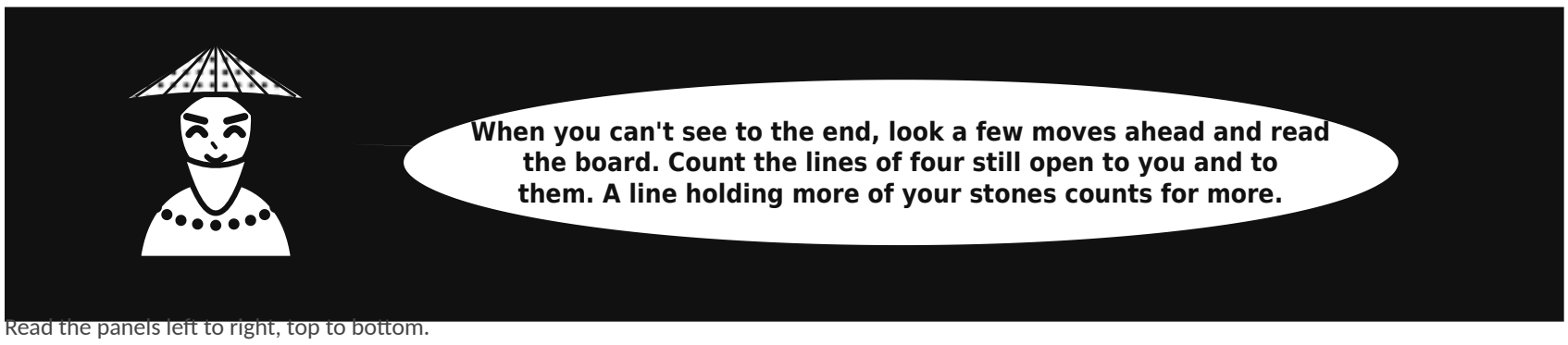












Read the panels left to right, top to bottom.

3 Scroll Box 3: Stone Drop

Find the move that guarantees a win, first by full search and then by looking four moves ahead.

Stone Drop rules

1. The board has 5 columns and 5 rows. Columns are numbered 0 to 4 from left to right. Rows are numbered 0 (bottom) to 4 (top).
2. The Pine team plays `X` and is the maximizing player. Vesper plays `0` and is the minimizing player. Players alternate.
3. A move names a column that is not full. The stone falls to the lowest empty cell of that column.
4. Four of one player's stones in a straight line wins: horizontal, vertical, or diagonal in either direction. The board has exactly 28 four-cell windows.
5. The game ends the moment someone completes four in a line, or when the board is full. A full board with no line of four is a draw.

Starting position

Shown with the top row (row 4) first. A dot is an empty cell. There are 7 `X`, 7 `0` and 11 empty cells, nobody has four in a line, and it is `X`'s turn.

```
.....
.0.X.
XX.0.
0X00.
0XXX0
```

Utility of a finished game

Result	Utility
`X` completes four in a line	1 + E
`0` completes four in a line	−(1 + E)
Full board, no line of four	0

E is the number of empty cells left when the game ends.

Search rules (used in Scroll Boxes 3 and 4)

- At every position, first test whether the game is over. If it is, return its value and generate no moves.
- Generate moves in the column order given for the run, skipping full columns.
- **Node:** every call of your search function counts as one node, including the starting position and every finished position.
- **Leaf:** a finished position reached by the search.
- **Best move:** at the root, the first column whose value is strictly better than every column tried before it.
- Do not use memoization or a transposition table.

Part A: Foresight Eye (full minimax)

Search to the end of the game with column order 0, 1, 2, 3, 4. Report the root value, the best move, the node count and the leaf count.

Part B: Reading the board (depth-limited minimax)

The root has depth 4, and each move lowers the depth by 1. Use column order 0, 1, 2, 3, 4.

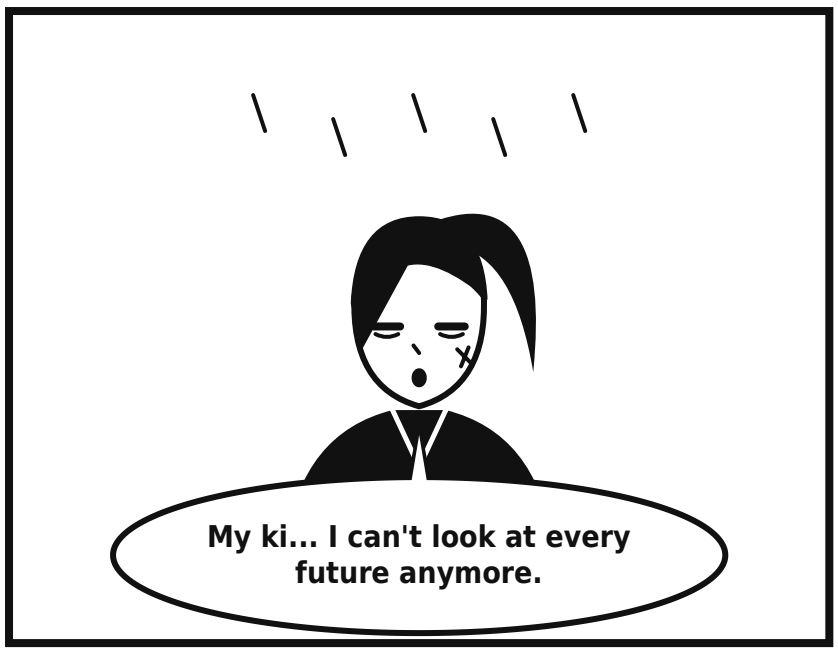
- A finished game returns its utility × 1000.
- An unfinished position at depth 0 returns its evaluation.
- **Evaluation:** go through all 28 windows. A window holding k ≥ 1 `X` stones and no `0` adds k^2. A window holding k ≥ 1 `0` stones and no `X` subtracts k^2. Every other window adds 0.

Report the root value, the best move and the node count.

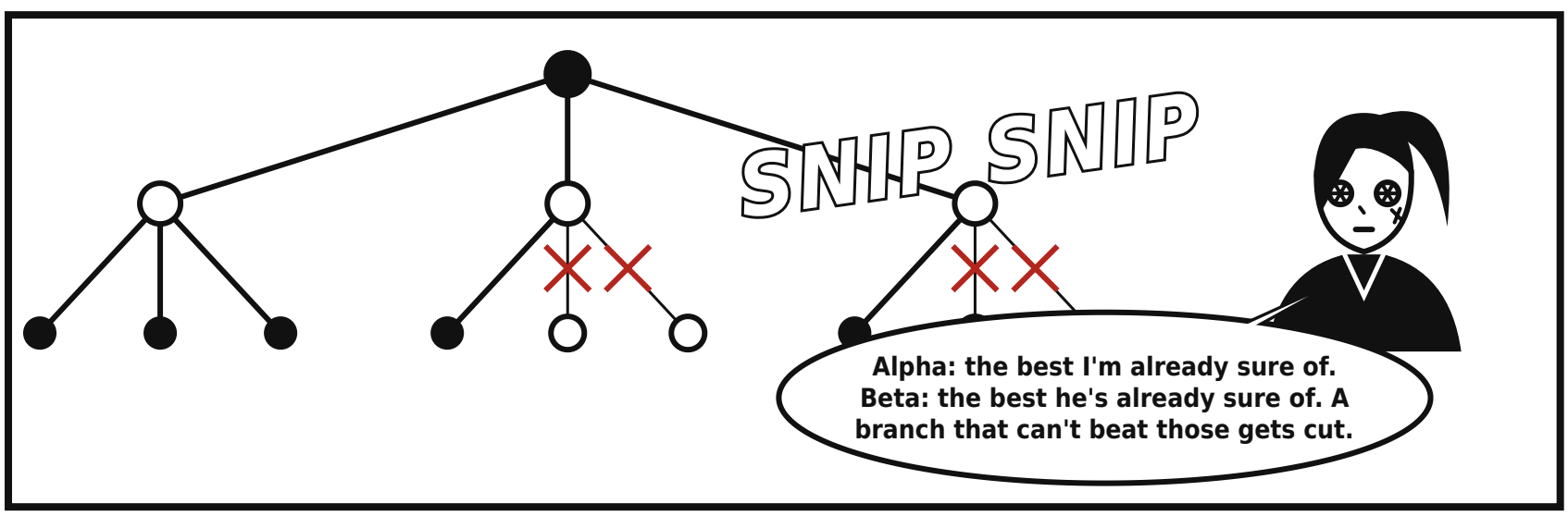
Report lines

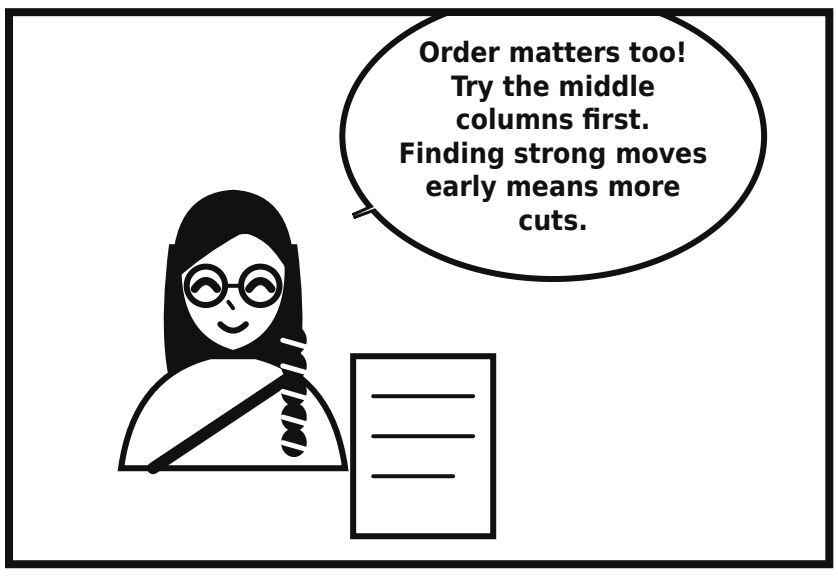
`MM_VALUE`, `MM_MOVE`, `MM_NODES`, `MM_LEAVES`, `DL_VALUE`, `DL_MOVE` and `DL_NODES`, in the format of Scroll Box 5.

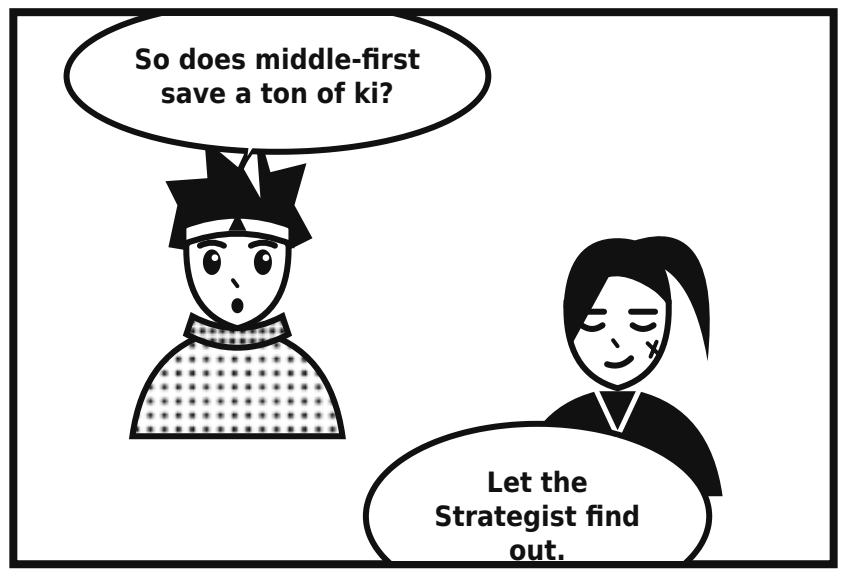
Chapter 4: Running on Empty

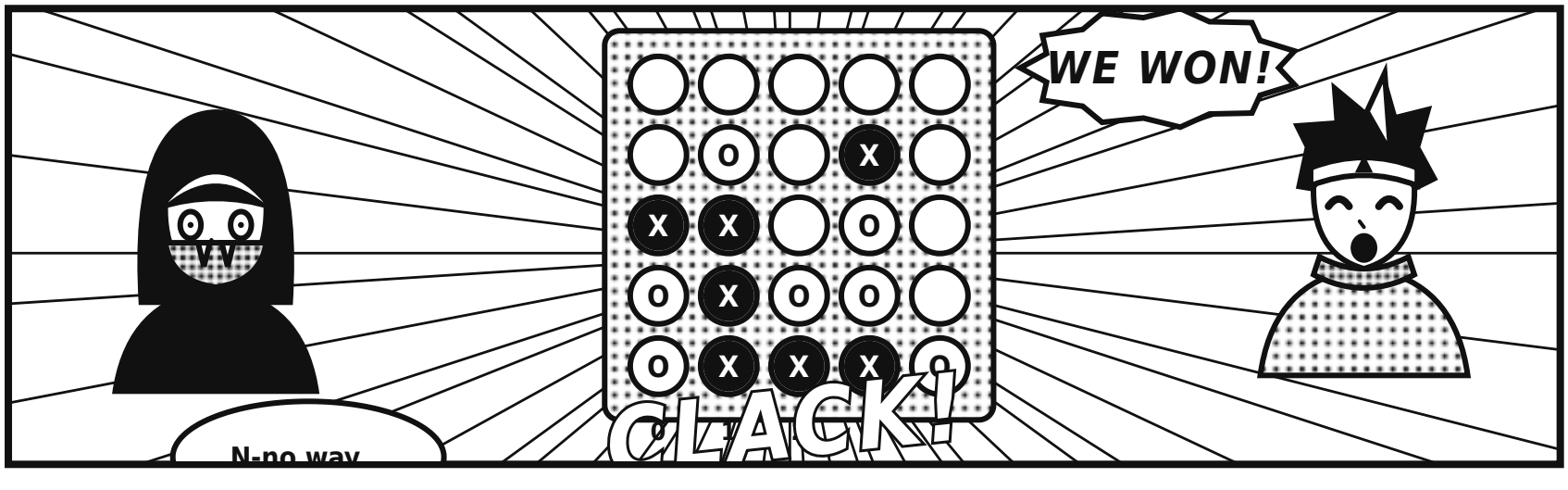












4 Scroll Box 4: Running on empty

Win the same duel while examining far fewer positions, and measure how move order changes the saving.

Pruning rules

The game, starting position, utilities, game-over test, node count and best-move rule are exactly as in Scroll Box 3. Only the search changes.

1. Call the search on the starting position with α = −∞ and β = +∞. Treat the root as an ordinary MAX node.
2. **MAX node:** set v = −∞. For each move in order, set v = max(v, value of the child searched with the current α and β). If v ≥ β, return v at once. Otherwise set α = max(α, v). After the last move, return v.
3. **MIN node:** set v = +∞. For each move in order, set v = min(v, value of the child searched with the current α and β). If v ≤ α, return v at once. Otherwise set β = min(β, v). After the last move, return v.
4. Return v itself when pruning (fail-soft), never α or β in its place.

Part A: Three Stone Drop runs

Each run uses its column order at every node of the tree. Full columns are skipped.

Run	Depth	Column order
AB1	Full depth, as in Scroll Box 3, Part A	0, 1, 2, 3, 4
AB2	Full depth, as in Scroll Box 3, Part A	2, 1, 3, 0, 4 (middle first)
AB3	Depth 4 with the evaluation, as in Scroll Box 3, Part B	0, 1, 2, 3, 4

For each run, report the root value, the best move and the node count. AB1 and AB2 must return the same value as Part A of Scroll Box 3, and AB3 the same value as Part B. If they do not, your pruning is wrong.

Part B: Master Hyo's trial tree

Master Hyo's trial is a game tree with four levels below the root. The root R is a MAX node, and levels alternate MIN, MAX, MIN. The 60 leaves L1 to L60 are terminal positions, and the number after each leaf is its terminal utility for MAX. Children are listed left to right.

```
R (MAX)
  A (MIN)
    A1 (MAX)
      A1a (MIN): L1=4, L2=-27, L3=-22
      A1b (MIN): L4=-28, L5=16, L6=13
      A1c (MIN): L7=2, L8=-20, L9=10
      A1d (MIN): L10=-15, L11=17, L12=-4
    A2 (MAX)
      A2a (MIN): L13=24, L14=-29, L15=12
      A2b (MIN): L16=-21, L17=-22
      A2c (MIN): L18=8, L19=0
    A3 (MAX)
      A3a (MIN): L20=-15, L21=22, L22=7
      A3b (MIN): L23=-29, L24=17
      A3c (MIN): L25=24, L26=4, L27=4
      A3d (MIN): L28=2, L29=-28
  B (MIN)
    B1 (MAX)
      B1a (MIN): L30=12, L31=5, L32=-10
      B1b (MIN): L33=24, L34=12, L35=-3
      B1c (MIN): L36=24, L37=14, L38=-18
      B1d (MIN): L39=26, L40=30
    B2 (MAX)
      B2a (MIN): L41=30, L42=9
      B2b (MIN): L43=25, L44=-3, L45=8
      B2c (MIN): L46=-21, L47=-27, L48=-24
  C (MIN)
    C1 (MAX)
      C1a (MIN): L49=5, L50=15, L51=19
      C1b (MIN): L52=-27, L53=-4
      C1c (MIN): L54=-7, L55=8
    C2 (MAX)
      C2a (MIN): L56=-10, L57=-24, L58=-4
      C2b (MIN): L59=30, L60=-4
```

Use exactly the fail-soft alpha-beta rules above, starting at R with α = −∞ and β = +∞. A leaf is **evaluated** when the search reaches it and reads its value.

1. **Left to right.** Visit children left to right at every node. Report the root value, the best move (A, B or C) using the best-move rule of Scroll Box 3, the number of leaves evaluated, and the evaluated leaves in the order they were reached.
2. **The window at B2.** In the left-to-right run, report the values of α and β that node B2 receives when the search calls it.
3. **Right to left.** Visit children right to left at every node and report the number of leaves evaluated.

Your program must compute these answers from the tree, and you should also trace the left-to-right run by hand. The debrief asks you to explain it.

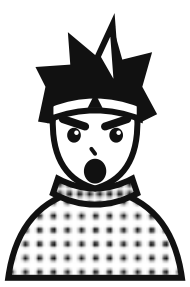
Report lines

`AB1`, `AB2` and `AB3`, each written as value, move and node count separated by commas, then `TREE_VALUE`, `TREE_MOVE`, `TREE_LR_LEAVES`, `TREE_LR_LIST`, `TREE_B2_WINDOW` and `TREE_RL_LEAVES`, in the format of Scroll Box 5.

Chapter 5: Debrief

5

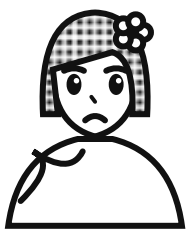




Why did I get stuck on that first hill? Show me the exact spot and everything around it!



Only some of my 20 fate scrolls reached the summit. Explain how the starting heat and cooling rate decide that. Try a starting heat of 40 and tell me what changes.



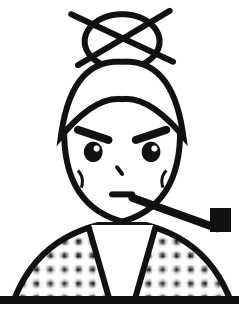
Why did my seeds stop at 234 when 241 was possible? Compare the two pouches seed by seed, and give me one way to fix my technique.



For every move that would have lost, show how I would have beaten you.



Compare the ki, the node counts, of every method. Why did middle-first barely help? And in Master Hyo's tree, why did right-to-left read so many fewer leaves?



STAMP!

Sora

Send your Owl Cipher and your debrief. The owls leave at sunset.

5 Scroll Box 5: The Owl Cipher

The exact report your program prints, and what the debrief must cover.

Owl Cipher report format

Print exactly these 36 lines in this order, replacing each placeholder with your computed value. Print nothing else.

```
=== OWL CIPHER ===
HC_FINAL: (r,c)
HC_VALUE: <integer>
HC_STEPS: <integer>
RR_BEST_VALUE: <integer>
RR_BEST_START: (r,c)
RR_TOTAL_STEPS: <integer>
SA_FINAL: (r,c)
SA_FINAL_VALUE: <integer>
SA_BEST_VALUE: <integer>
SA_ACCEPTED: <integer>
SA_SUCCESS_1_TO_20: <integer>
GA_GEN_BEST: g0,g10,g20,g30,g40,g50
GA_BEST_CHROMOSOME: <15 bits>
GA_BEST_FITNESS: <integer>
GA_FIRST_GEN: <integer>
GA_FINAL_AVG: <number with 2 decimals>
OPT_FITNESS: <integer>
OPT_CHROMOSOME: <15 bits>
MM_VALUE: <integer>
MM_MOVE: <column>
MM_NODES: <integer>
MM_LEAVES: <integer>
DL_VALUE: <integer>
DL_MOVE: <column>
DL_NODES: <integer>
AB1: value,move,nodes
AB2: value,move,nodes
AB3: value,move,nodes
TREE_VALUE: <integer>
TREE_MOVE: A, B or C
TREE_LR_LEAVES: <integer>
TREE_LR_LIST: <leaf names>
TREE_B2_WINDOW: (alpha,beta)
TREE_RL_LEAVES: <integer>
MISSION_STATUS: SUCCESS or FAILED
```

- A cell is written `(r,c)` with no spaces, for example `(3,5)`.
- Lists such as `GA_GEN_BEST`, `AB1` and `TREE_LR_LIST` use commas with no spaces, for example `100,120,140,150,150,160` or `L5,L6,L9`.
- `TREE_B2_WINDOW` is written `(alpha,beta)` with no spaces, for example `(3,12)`. Write infinities as `-inf` and `inf`.
- Each line has exactly one space after the colon and no trailing spaces.
- `MISSION_STATUS` is `SUCCESS` if `MM_VALUE` is greater than 0, and `FAILED` otherwise.

Debrief report

Submit `debrief.pdf`, at most four pages, answering the five questions asked in Chapter 5 by Kaito, Gennai, Rin, Vesper and Jun. Use your program's output as evidence. The extra runs those questions ask for are not part of the Owl Cipher report.